

1. Balanced Parentheses and Delimiters

- Write a C program that checks whether a string of parentheses is balanced or not using stack.
- An opening symbol that has a corresponding closing symbol is considered balanced parentheses, where the parentheses are correctly nested and the opening and closing symbols are the same.
- Valid balanced parentheses:
 - `[[{{(())}}]]`
 - `[][][](){}`
- Invalid balanced parentheses:
 - `([ )]`
 - `(( ( ) ] ) )`
 - `[ { ( ) ]`

```
Input an expression in parentheses: {[ ]}
The expression is not balanced.
-----
Input an expression in parentheses: (( ( )) )
The expression is balanced.
-----
Input an expression in parentheses: ( )
The expression is not balanced.
-----
Input an text of parentheses: ([ ]){ }[[ ( ) { } ]{ } ]
The expression is balanced.
-----
Input an expression in parentheses: [ ( ) ]
The expression is not balanced.
```

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define MAX_SIZE 100
6
7  // Global variables for stack and top
8  char stack[MAX_SIZE];
9  int top = -1;
10
11 // Function to push a character onto the stack
12 void push(char data) {
13     if (top == MAX_SIZE - 1) {
14         printf("Overflow stack!\n");
15         return;
16     }
17     top++;
18     stack[top] = data;
19 }
20
```

```
21 // Function to pop a character from the stack
22 char pop() {
23     if (top == -1) {
24         printf("Empty stack!\n");
25         return ' ';
26     }
27     char data = stack[top];
28     top--;
29     return data;
30 }
31
32 // Function to check if two characters form a matching pair of parentheses
33 int is_matching_pair(char char1, char char2) {
34     if (char1 == '(' && char2 == ')') {
35         return 1;
36     } else if (char1 == '[' && char2 == ']') {
37         return 1;
38     } else if (char1 == '{' && char2 == '}') {
39         return 1;
40     } else {
41         return 0;
42     }
43 }
44
```

```
45 // Function to check if the expression is balanced
46 int isBalanced(char* text) {
47     int i;
48     for (i = 0; i < strlen(text); i++) {
49         if (text[i] == '(' || text[i] == '[' || text[i] == '{') {
50             push(text[i]);
51         } else if (text[i] == ')' || text[i] == ']' || text[i] == '}') {
52             if (top == -1) {
53                 return 0; // If no opening bracket is present
54             } else if (!is_matching_pair(pop(), text[i])) {
55                 return 0; // If closing bracket doesn't match the last opening bracket
56             }
57         }
58     }
59     if (top == -1) {
60         return 1; // If the stack is empty, the expression is balanced
61     } else {
62         return 0; // If the stack is not empty, the expression is not balanced
63     }
64 }
65
```

```
66 // Main function
67 int main() {
68     char text[MAX_SIZE];
69     printf("Input an expression in parentheses: ");
70     scanf("%s", text);
71
72     // Check if the expression is balanced or not
73     if (isBalanced(text)) {
74         printf("The expression is balanced.\n");
75     } else {
76         printf("The expression is not balanced.\n");
77     }
78     return 0;
79 }
```

2 . Middle Element Operations in Stack

- Write a C program to implement a stack that supports push, pop, get middle, and delete middle elements.
- *Expected Output:*
- Stack elements: 88 15 26 32 23
- Middle element: 26

- Delete the middle element of the said stack:
- Stack elements: 88 15 32 23
- Middle element: 15

- Delete the middle element of the said stack:
- Stack elements: 88 32 23
- Middle element: 32

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Node structure to represent elements in the stack
5  struct Node {
6      int data;
7      struct Node* next;
8      struct Node* prev;
9  };
10
11 // Stack structure with top, middle pointers, and a counter to track elements
12 struct Stack {
13     struct Node* top;
14     struct Node* middle;
15     int ctr;
16 };
17
```

```
18 // Function to push an element onto the stack
19 void push(struct Stack* stack, int data) {
20     // Create a new node
21     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
22     newNode->data = data;
23     newNode->prev = NULL;
24     newNode->next = stack->top;
25
26     // Update the middle pointer based on the number of elements
27     if (stack->ctr == 0) {
28         stack->middle = newNode;
29     } else if (stack->ctr % 2 == 0) {
30         stack->middle = stack->middle->prev;
31     }
32
33     // Update the previous pointer of the top node
34     if (stack->top != NULL) {
35         stack->top->prev = newNode;
36     }
37
38     // Update the top pointer and increment the counter
39     stack->top = newNode;
40     stack->ctr++;
41 }
42
```



```
43 // Function to pop an element from the stack
44 int pop(struct Stack* stack) {
45     if (stack->ctr == 0) {
46         printf("Stack is underflow\n");
47         return -1;
48     }
49
50     // Pop the top node
51     struct Node* top_node = stack->top;
52     int data = top_node->data;
53
54     stack->top = top_node->next;
55
56     if (stack->top != NULL) {
57         stack->top->prev = NULL;
58     }
59
60     free(top_node);
61
62     // Update the middle pointer based on the number of elements
63     if (stack->ctr % 2 == 1) {
64         stack->middle = stack->middle->next;
65     }
66
67     stack->ctr--;
68
69     return data;
70 }
71
```

```

72 // Function to get the middle element of the stack
73 int get_middle(struct Stack* stack) {
74     if (stack->ctr == 0) {
75         printf("Stack is empty\n");
76         return -1;
77     }
78
79     return stack->middle->data;
80 }
81
82 // Function to delete the middle element of the stack
83 void deleteMiddle(struct Stack* stack) {
84     if (stack->ctr == 0) {
85         printf("Stack is empty\n");
86         return;
87     }
88
89     // Get the middle node
90     struct Node* middle_node = stack->middle;
91
92     // Update the pointers based on the number of elements
93     if (stack->ctr == 1) {
94         stack->top = NULL;
95         stack->middle = NULL;
96     } else {
97         stack->middle = stack->ctr % 2 == 0 ? stack->middle->next : stack->middle->prev;
98         middle_node->prev->next = middle_node->next;
99         middle_node->next->prev = middle_node->prev;
100     }
101
102     free(middle_node);
103     stack->ctr--;
104 }
105

```

```

106 // Function to print the elements of the stack
107 void printStack(struct Stack* stack) {
108     if (stack->ctr == 0) {
109         printf("Stack is empty\n");
110         return;
111     }
112
113     struct Node* current = stack->top;
114     while (current != NULL) {
115         printf("%d ", current->data);
116         current = current->next;
117     }
118     printf("\n");
119 }
120
121 // Main function
122 int main() {
123     struct Stack stack = {NULL, NULL, 0}; // Initialize the stack
124
125     // Push elements onto the stack
126     push(&stack, 23);
127     push(&stack, 32);
128     push(&stack, 26);
129     push(&stack, 15);
130     push(&stack, 88);
131
132     // Display the stack elements
133     printf("Stack elements: ");
134     printStack(&stack);
135
136     // Display the middle element
137     printf("Middle element: %d\n", get_middle(&stack));
138

```

```
139 // Delete the middle element
140 printf("\nDelete the middle element of the stack:\n");
141 deleteMiddle(&stack);
142
143 // Display the updated stack and middle element
144 printf("\nStack elements: ");
145 printStack(&stack);
146 printf("Middle element: %d\n", get_middle(&stack));
147
148 // Delete another middle element
149 printf("\nDelete the middle element of the stack:\n");
150 deleteMiddle(&stack);
151
152 // Display the updated stack and middle element
153 printf("\nStack elements: ");
154 printStack(&stack);
155 printf("Middle element: %d\n", get_middle(&stack));
156
157 return 0;
158 }
```

3. Queue Reverse Order

- Write a C program to reverse the elements of a queue.
- *Expected Output:*
- Queue elements are:
- 1 2 3 4 5
- Reverse Queue, elements are:
- 5 4 3 2 1
- Add two elements to the said queue:
- Queue elements are:
- 5 4 3 2 1 100 200
- Reverse Queue, elements are:
- 200 100 1 2 3 4 5

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #define MAX_SIZE 100
4
5  // Define a structure for the queue
6  typedef struct {
7      int items[MAX_SIZE]; // Array to store elements of the queue
8      int front; // Index of the front element
9      int rear; // Index of the rear element
10 } Queue;
11
12 // Function to create an empty queue
13 Queue* createQueue() {
14     Queue* q = (Queue*)malloc(sizeof(Queue)); // Allocate memory for the queue
15     q->front = -1; // Initialize front to -1
16     q->rear = -1; // Initialize rear to -1
17     return q; // Return the created queue
18 }
19
20 // Function to check if the queue is empty
21 int isEmpty(Queue* q) {
22     return q->rear == -1; // Return 1 if rear is -1, indicating an empty queue
23 }
24
25 // Function to check if the queue is full
26 int isFull(Queue* q) {
27     return q->rear == MAX_SIZE - 1; // Return 1 if rear is at the maximum size, indicating a full queue
28 }
29
```

```
30 // Function to add an element to the queue
31 void enqueue(Queue* q, int value) {
32     if (isFull(q)) { // Check if the queue is full
33         printf("Queue is full\n"); // Print error message if the queue is full
34     } else {
35         if (q->front == -1) { // Check if front is -1 (indicating an empty queue)
36             q->front = 0; // Set front to 0
37         }
38         q->rear++; // Increment rear
39         q->items[q->rear] = value; // Add the element to the rear of the queue
40     }
41 }
42
43 // Function to remove an element from the queue
44 int dequeue(Queue* q) {
45     int item;
46     if (isEmpty(q)) { // Check if the queue is empty
47         printf("Queue is empty\n"); // Print error message if the queue is empty
48         item = -1; // Return -1 as an indication of an empty queue
49     } else {
50         item = q->items[q->front]; // Get the element at the front of the queue
51         q->front++; // Increment front
52         if (q->front > q->rear) { // Check if front has crossed rear (indicating an empty queue)
53             q->front = q->rear = -1; // Reset front and rear to -1
54         }
55     }
56     return item; // Return the removed element
57 }
58
```

```
59 // Function to reverse the elements in a queue
60 void reverse_queue(Queue* q) {
61     int temp[MAX_SIZE]; // Temporary array to store elements
62     int top = -1; // Initialize top of the temporary array
63
64     // Dequeue elements from the queue and store them in the temporary array
65     while (!isEmpty(q)) {
66         temp[++top] = dequeue(q);
67     }
68
69     // Enqueue elements from the temporary array back to the queue, effectively reversing the order
70     while (top != -1) {
71         enqueue(q, temp[top--]);
72     }
73 }
74
75 // Function to display the elements in the queue
76 void display(Queue* q) {
77     if (isEmpty(q)) { // Check if the queue is empty
78         printf("Queue is empty\n"); // Print message if the queue is empty
79     } else {
80         // Loop through the elements of the queue and print each element
81         for (int i = q->front; i <= q->rear; i++) {
82             printf("%d ", q->items[i]); // Print the element at index i
83         }
84         printf("\n"); // Print a newline after displaying all elements
85     }
86 }
87
```



```
88 // Main function to test the program
89 int main() {
90     Queue* q = createQueue(); // Create an empty queue
91
92     // Add elements to the queue
93     enqueue(q, 1);
94     enqueue(q, 2);
95     enqueue(q, 3);
96     enqueue(q, 4);
97     enqueue(q, 5);
98
99     // Display the elements in the queue
100    printf("Queue elements are:\n");
101    display(q);
102
103    // Reverse the elements in the queue
104    printf("Reverse Queue, elements are:\n");
105    reverse_queue(q);
106
107    // Display the elements in the reversed queue
108    display(q);
109}
```

```
110 // Add two elements to the queue
111 enqueue(q, 100);
112 enqueue(q, 200);
113
114 printf("Add two elements to the said queue:\n");
115 printf("Queue elements are:\n");
116 display(q);
117
118 // Reverse the elements in the queue
119 printf("Reverse Queue, elements are:\n");
120 reverse_queue(q);
121 display(q);
122
123 return 0; // Return from the main function
124 }
125
```

4. Delete nth Element in Queue

- Write a C program to delete the n^{th} element of a queue.
- Firstly, this code checks if the queue is empty or if the position to delete is invalid. If the position is valid, it deletes the first $n-1$ elements, then deletes the n^{th} element by calling the `dequeue()` function.
- *Expected Output:*
- Insert some elements into the queue:
- Queue elements are: 1 2 3 4 5
- Delete the 7th element of the said queue:
- Error: Invalid position
- Delete the 3rd element of the said queue:
- Queue elements are: 4 5

```
1  #include <stdio.h>
2  #define MAX_SIZE 100
3
4  int queue[MAX_SIZE]; // Array to store elements of the queue
5  int front = -1, back = -1; // Initialize front and back pointers
6
7  // Function to insert an element into the queue
8  void enqueue(int item) {
9      if (back == MAX_SIZE - 1) { // Check if the queue is full
10         printf("Error: Queue is full\n"); // Print error message if the queue is full
11         return;
12     }
13     if (front == -1) { // Check if the queue is empty
14         front = 0; // If empty, set front to 0
15     }
16     back++; // Increment the back pointer
17     queue[back] = item; // Add the item to the queue
18 }
19
20 // Function to remove an element from a queue
21 int dequeue() {
22     if (front == -1 || front > back) { // Check if the queue is empty
23         printf("Error: Queue is empty\n"); // Print error message if the queue is empty
24         return -1; // Return -1 to indicate an empty queue
25     }
26     int item = queue[front]; // Get the front element of the queue
27     front++; // Move the front pointer to the next element
28     return item; // Return the removed element
29 }
30
```

```
31 // Function to delete the nth element of a queue
32 void delete_nth(int n) {
33     if (front == -1 || front > back) { // Check if the queue is empty
34         printf("Error: Queue is empty\n"); // Print error message if the queue is empty
35         return;
36     }
37     if (n <= 0 || n > back - front + 1) { // Check if the position is valid
38         printf("Error: Invalid position\n"); // Print error message if the position is invalid
39         return;
40     }
41
42     // Delete the first n - 1 elements
43     for (int i = 1; i < n; i++) {
44         dequeue(); // Call dequeue to remove elements until reaching the nth position
45     }
46
47     // Delete the nth element
48     dequeue(); // Call dequeue to remove the nth element
49 }
50
51 // Function to display the elements in the queue
52 void display_elements() {
53     printf("Queue elements are: "); // Print message to indicate queue elements
54     for (int i = front; i <= back; i++) { // Loop through the elements of the queue
55         printf("%d ", queue[i]); // Print the current element
56     }
57     printf("\n"); // Print a newline after displaying all elements
58 }
59
```

```
60  int main() {
61      // Insert some elements into the queue.
62      printf("Insert some elements into the queue:\n");
63      enqueue(1);
64      enqueue(2);
65      enqueue(3);
66      enqueue(4);
67      enqueue(5);
68      display_elements(); // Display the elements in the queue
69      // Delete the 7th element.
70      printf("\nDelete the 7th element of the said queue:\n");
71      delete_nth(7); // Attempt to delete the 7th element
72      // Delete the 3rd element.
73      printf("\nDelete the 3rd element of the said queue:\n");
74      delete_nth(3); // Attempt to delete the 3rd element
75      display_elements(); // Display the elements in the queue after deletion
76      return 0; // Return 0 to indicate successful execution
77  }
```